

GNU nano 7.2

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handle_signal(int sig)
{
    printf("\nSIGINT received!\n");
}
int main()
{
    signal(SIGINT, handle_signal);
    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

```
SIGINT received!
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

2.

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
int main()
{
while (1)
{
printf("Program is running...\n");
sleep(2);
}
return 0;
}|
```

```
sravya@Sravya1955:~/os$ gcc signal2.c -o
sravya@Sravya1955:~/os$ ./signal2
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
sravya@Sravya1955:~/os$ |
```

3.

```

GNU nano 7.2
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handle_sigchld(int sig)
{
printf("SIGCHLD received: Child has terminated.\n");
}
int main()
{
signal(SIGCHLD, handle_sigchld);
pid_t pid = fork();
if (pid == 0)
{
// Child
printf("Child is running...\n");
sleep(3);
printf("Child is terminating...\n");
}
else
{
// Parent
printf("Parent is waiting...\n");
wait(NULL);
printf("Parent finished.\n");
}
return 0;
}

```

```

sravya@Sravya1955:~/os$ nano signal3.c
sravya@Sravya1955:~/os$ gcc signal3.c -o signal3
sravya@Sravya1955:~/os$ ./signal3
Parent is waiting...
Child is running...
Child is terminating...
SIGCHLD received: Child has terminated.
Parent finished.

```

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
int main()
{
    int fd[2];
    pipe(fd);
    close(fd[0]); // Close read end
    printf("Writing to pipe...\n");
    write(fd[1], "Hello", 5);
    printf("Write completed.\n");
    return 0;
}
```

4.

```
sravya@Sravya1955:~/os$ nano signal4.c
sravya@Sravya1955:~/os$ gcc signal4.c -o signal4
sravya@Sravya1955:~/os$ ./signal4
Writing to pipe
```

5.

GNU nano 7.2

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handler(int sig)
{
printf("Child received SIGUSR1\n");
}
int main()
{
pid_t pid = fork();
if (pid == 0)
{
// Child
signal(SIGUSR1, handler);
printf("Child waiting...\n");
while (1)
{
sleep(1);
}
}
else
{
// Parent
sleep(2);
printf("Parent sending signal...\n");
kill(pid, SIGUSR1);
sleep(2);
kill(pid, SIGTERM);
wait(NULL);
}
return 0;
}
```

```
sravya@Sravya1955:~/os$ nano signal5.c
sravya@Sravya1955:~/os$ gcc signal5.c -o signal5
sravya@Sravya1955:~/os$ ./signal5
Child waiting...
Parent sending signal...
Child received SIGUSR1
```

6.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
int main()
{
printf("Program started\n");
abort();
printf("Program ended\n");
return 0;
}
```

```
Child received SIGUSR1
sravya@Sravya1955:~/os$ nano signal5.c
sravya@Sravya1955:~/os$ nano signal6.c
sravya@Sravya1955:~/os$ gcc signal6.c -o signal6
sravya@Sravya1955:~/os$ ./signal6
Program started
Aborted (core dumped)
```

7.

```
GNU nano 7.2
#include <stdio.h>
int main()
{
int *p = NULL;
printf("Before error\n");
*p = 10;
printf("After error\n");
return 0;
}
|
```

```
sravya@Sravya1955:~/os$ nano signal7.c
sravya@Sravya1955:~/os$ gcc signal7.c -o signal7
sravya@Sravya1955:~/os$ ./signal7
Before error
Segmentation fault (core dumped)
```